

1. 相关消息类型
2. 使用场景
3. ROS协议约束
- 3.1 服务接口
- 3.1.1 子状态注册
- 3.1.2 子状态更新
- 3.1.3 主动查询当前状态
- 3.2 Topic接口
- 3.2.1 主状态变化时广播
- 3.2.2 低频播报当前状态
- 3.2.3 需重点关注状态的高频播报

👤 李向远👤 李雨薇

|

4月8日创建

版本	日期	内容	修订人
v0.1	2025.4.11	初版协议	👤 李雨薇

1. 相关消息类型

- protocol/msg/RobotState.msg

```
1 # 主要状态
2 uint8 STATE_STANDBY           = 1
3 uint8 STATE_RUNNING           = 2
4 uint8 STATE_OTA               = 3
5 uint8 STATE_EMERGENCYSTOP     = 4
6 uint8 STATE_SHUTDOWN          = 5
7
8 # 状态ID, 使用上面的编号
9 uint8 id
10 # 状态的文本标签
11 string label
```

- protocol/msg/SubState.msg

```
1 string fsm      # 子状态键名 (如"arm", "power")
2 int status      # 子状态值 (例如 "0")
```

- protocol/msg/RobotTransition.msg

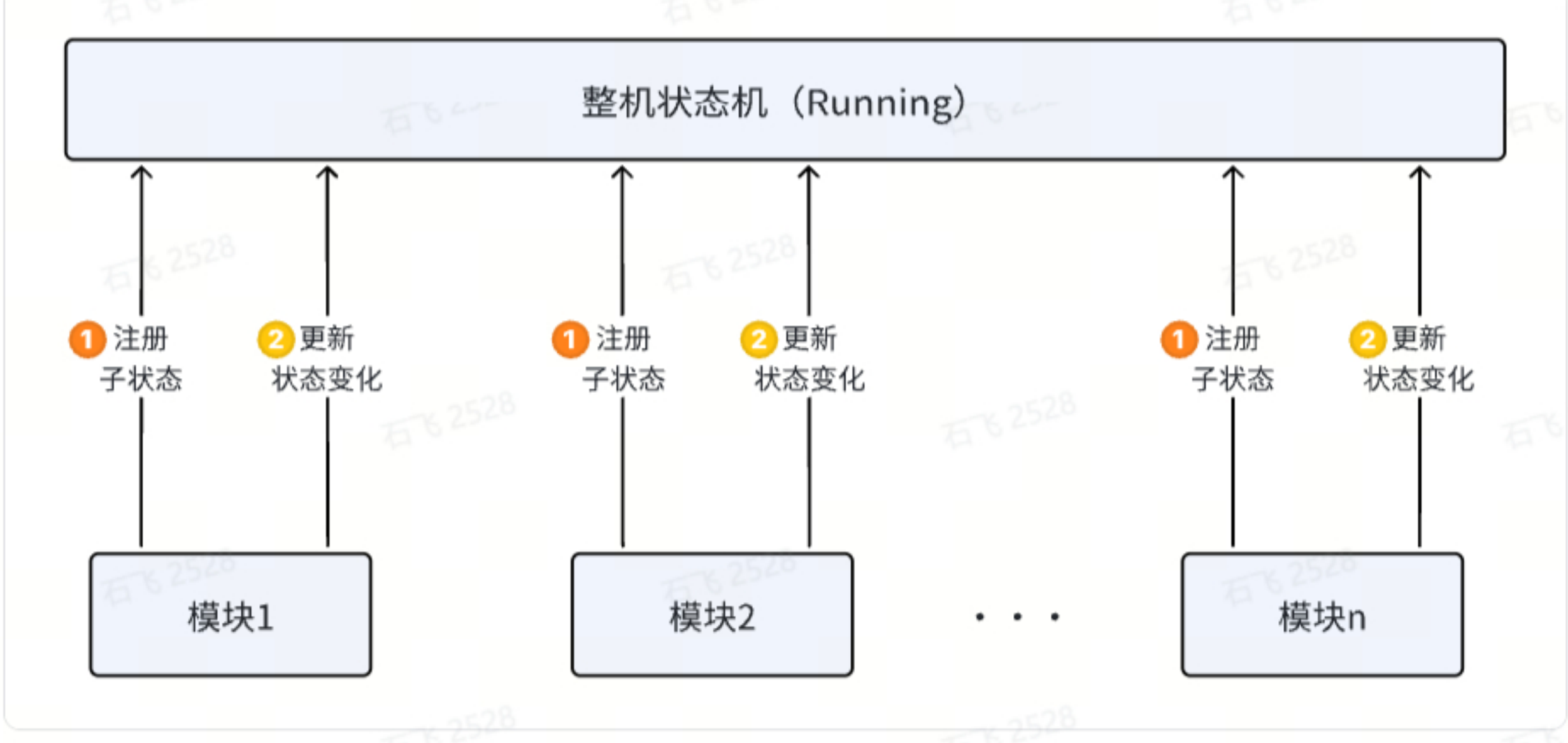
```
1 # [0-9], 公开可调用的转换
2 uint8 TRANSITION_CREATE           = 1 # 启动到standby状态
3 uint8 TRANSITION_ACTIVATE        = 2 # Standby -> Running
4 uint8 TRANSITION_SHUTDOWN        = 3 # Running -> Shutdown
5 uint8 TRANSITION_UPDATEPREPARE   = 4 # Standby -> OTA
6 uint8 TRANSITION_UPDATECOMPLETE = 5 # OTA -> Standby
7 uint8 TRANSITION_ESTOP            = 6 # Running -> Emergency-Stop
8 uint8 TRANSITION_RESTORE         = 7 # Emergency-Stop -> Running
9
10 # 转换ID, 使用上面的编号
11 uint8 id
12
13 # 转换的文本标签
14 string label
```

- protocol/msg/TransitionDescription.msg

```
1 # RobotTransition格式, 记录了转换的 ID 和标签
2 RobotTransition transition
3
4 # 当前状态, 从该状态开始过渡
5 RobotState start_state
6
7 # 此过渡的目标状态
8 RobotState goal_state
```

2. 使用场景

- 对于各模块而言提供接口注册



3. ROS协议约束

3.1 服务接口

3.1.1 子状态注册

- 消息类型: protocol/srv/RegisterSubState.srv
- 服务名称: /system/register_sub_state

```
1 string operation      # 操作类型 ("REGISTER"/"UNREGISTER")
2 string requester      # 发起者
3 string fsm            # 状态键名 (如"arm", "power")
4 uint[] state          # 合法状态值数组
5 string[] state_name   # 合法状态值对应含义数组
6 ---
7 # 响应结果
8 bool success
9 string error_msg      # 失败原因
```

3.1.2 子状态更新

- 消息类型: protocol/srv/UpdateSubState.srv
- 服务名称: /system/update_sub_state

```
1 string requester      # 发起者
2 string fsm            # 状态键名 (如"arm", "power")
3 int new_value         # 状态值
4 ---
5 # 响应结果
6 bool success
7 string error_msg      # 失败原因
```

3.1.3 主动查询当前状态

- 消息类型: protocol/srv/GetState.srv
- 服务名称: /system/get_state

```
1 ---
2 # 整机状态
3 RobotState current_state
4
5 # 维护的子状态数组
6 SubState[] sub_status
```

3.2 Topic接口

3.2.1 主状态变化时广播

- 消息类型: protocol/msg/TransitionEvent.msg
- Topic 名称: /system/transition_event

```
1 # 时间戳
2 uint64 timestamp
3
4 # 转换事件的 ID 和标签
5 RobotTransition transition
6
7 # 转换事件的起始状态
8 RobotState start_state
9
10 # 转换事件的结束状态
11 RobotState end_state
```

除了“查询”以外系统提供主动上报状态变化的接口, 通过发布一个名为 `/system/transition_event` 的 ROS 2 Topic, 每当主状态机完成一次状态切换, 都会自动发布一条 `TransitionEvent` 类型的消息。其中 `end_state` 表示的是状态切换后的真实状态。

3.2.2 低频播报当前状态

- 消息类型: protocol/msg/CurrentState.msg
- Topic 名称: /system/current_state

```
1 # 时间戳
2 uint64 timestamp
3
4 # 整机状态
5 RobotState current_state
6
7 # 维护的子状态数组
8 SubState[] sub_status
```

除了只在主状态发生变化时发布的Topic以外还提供一个名为 `/system/current_state` 的 ROS 2 Topic 周期性广播整机的实时状态, 消息类型为 `CurrentState`。

3.2.3 需重点关注状态的高频播报

- 消息类型: protocol/msg/ImportantSubState.msg
- Topic 名称: /system/important_sub_state

```
1 # 时间戳
2 uint64 timestamp
3
4 # 发起者
5 string requester
6
7 # 子状态的键, 例如 "arm", "battery", "camera"
8 string key
9
10 # 子状态具体值
11 int value
```

对于例如低电量状态等各组件需要重点关注的紧急状态, 系统会发布频次更高的Ros Topic进行广播, 消息类型为 `ImportantSubState`。

反向引用 (1)🔗

本文引用 (0)

👤 关系图

- ▶
- 📘 机器人整机状态机技术方案



真诚点赞, 手留余香

输入评论

